



C++ LOOPS MASTER CHEAT SHEET

Target: DSA Marathon | Placement Preparation | Quick Revision

1. Overview of Loops

What is a Loop?: A control flow structure that repeatedly executes a block of code while a specified boolean condition remains true.

Why Use Loops?: Avoids code duplication, reduces program size, improves code maintainability, and processes data dynamically.

Where Used?: Array/Vector traversal, algorithm execution (Binary Search, Sorting, Graph Traversals), input processing, and continuous game/system loops.

2. for Loop

Definition & Purpose

A pre-tested, count-controlled loop structure used when the exact number of iterations is known prior to loop execution.

Syntax

```
C++
#include <iostream>
using namespace std;
int main() {
    for (initialization; condition; update)
    {
        // Loop Body Statements
    }
    return 0;
}
```

Syntax Explanation

- initialization: Runs once before entering the loop (e.g., `int i = 1;`).
- condition: Evaluated before every iteration. If true, the body executes; if false, the loop terminates.
- update: Runs after each iteration body finishes (e.g., `i++`).

Executable Example

```
C++
#include <iostream>
using namespace std;
int main() {
    // Print numbers 1 to 3
}
```

```
for (int i = 1; i <= 3; i++)
{
    cout << "Iteration: " << i << endl;
}
return 0;
}
```

Output: Iteration: 1 Iteration: 2 Iteration: 3

3. while Loop

Definition & Purpose

A pre-tested condition-controlled loop that runs as long as the specified condition remains true. Use this when the number of iterations is not known in advance.

Syntax

```
C++
#include <iostream>
using namespace std;
int main() {
    initialization;
    while (condition)
    {
        // Statements
        update;
    }
    return 0;
}
```

Syntax Explanation

Checks the condition before entering the loop. If the initial condition evaluates to false, the loop body executes 0 times.

Executable Example

```
C++
#include <iostream>
using namespace std;
int main() {
    int count = 1;
    // Run while count is less than or equal to 3
    while (count <= 3)
    {
        cout << "Count: " << count << endl;
        count++;
    }
    return 0;
}
```

Output: Count: 1 Count: 2 Count: 3

4. do-while Loop

Definition & Purpose

A post-tested loop structure where the condition is evaluated after execution. It guarantees the loop body runs at least once regardless of whether the initial condition is true or false.

Syntax

```
C++
#include <iostream>
using namespace std;
int main() {
    initialization;
    do
    {
        // Statements
        update;
    }
    while (condition); // Trailing semicolon required
    return 0;
}
```

CRUCIAL NOTE

A trailing semicolon ; is strictly required after while(condition);.

Executable Example

```
C++
#include <iostream>
using namespace std;
int main() {
    int val = 10;
    // Executes once even though (10 < 5) is false
    do
    {
        cout << "Executing once. Value: " << val << endl;
        val++;
    }
    while (val < 5);
    return 0;
}
```

Output: Executing once. Value: 10

5. break Statement

Definition & Purpose

An unconditional jump statement used to terminate the loop or switch block immediately.

Syntax

C++

```
break;
```

Executable Example

C++

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 1; i <= 5; i++)
    {
        if (i == 3)
        {
            break; // Exits loop completely when i reaches 3
        }
        cout << i << " ";
    }
    return 0;
}
```

Output: 1 2

6. continue Statement

Definition & Purpose

A jump statement that skips the remaining statements inside the current iteration and jumps directly to the update step / condition check for the next iteration.

Syntax

C++

```
continue;
```

Executable Example

C++

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 1; i <= 5; i++)
    {
        if (i == 3)
        {
            continue; // Skips printing 3
        }
        cout << i << " ";
    }
    return 0;
}
```

Output: 1 2 4 5

8. Common Beginner Errors to Avoid

1. Accidental Semicolon After Loop Header

✗ **Incorrect:**

```
C++
for (int i = 0; i < 5; i++); // Semicolon terminates header!
{
    cout << i;
}
```

✓ **Correct:**

```
C++
for (int i = 0; i < 5; i++)
{
    cout << i;
}
```

2. Missing Variable Updates (Infinite Loop)

✗ **Incorrect:**

```
C++
int i = 0;
while (i < 5)
{
    cout << i << endl; // Missing i++, runs infinitely
}
```

✓ **Correct:**

```
C++
int i = 0;
while (i < 5)
{
    cout << i << endl;
    i++;
}
```

📌 Core Loop Summaries

- for Loop: Best when the number of iterations is known in advance (e.g., array/vector traversals).
- while Loop: Best when iterations depend on dynamic runtime conditions (e.g., dynamic input streams, linked lists).
- do-while Loop: Guarantees execution at least once before evaluating the condition (e.g., menu systems, input validation).
- break Statement: Terminates the loop or switch block immediately, regardless of the loop's own condition.
- continue Statement: Skips the rest of the current iteration and jumps straight to the next one.

End of C++ Loops Master Cheat Sheet



NextGen Data Minds